

HOW LARGE LANGUAGE MODELS ARE BUILT

From Raw Data to Intelligent AI Systems

Ruthran Raghavan

Chief AI Scientist

HERE AND NOW AI – Artificial Intelligence Research Institute

www.ruthranraghavan.com

NLP

Transformers

LLMs

Deep Learning

AI Engineering

Why Large Language Models Matter

- LLMs mark a turning point in AI — systems now understand, generate, and reason with human language at scale.
- Generative AI boom: ChatGPT reached 100M users in 60 days (fastest product in history).
- Applications span code generation, medical diagnosis, education, legal analysis, creative writing, and robotics.
- Enterprise adoption: 75%+ of Fortune 500 companies are piloting generative AI solutions (McKinsey, 2024).
- Economic impact: AI projected to add \$15.7 trillion to global GDP by 2030 (PwC).
- Foundational technology: LLMs are the backbone of agents, RAG systems, and multimodal AI.

1T+

Tokens in GPT-4 training data

175B

GPT-3 parameters (2020)

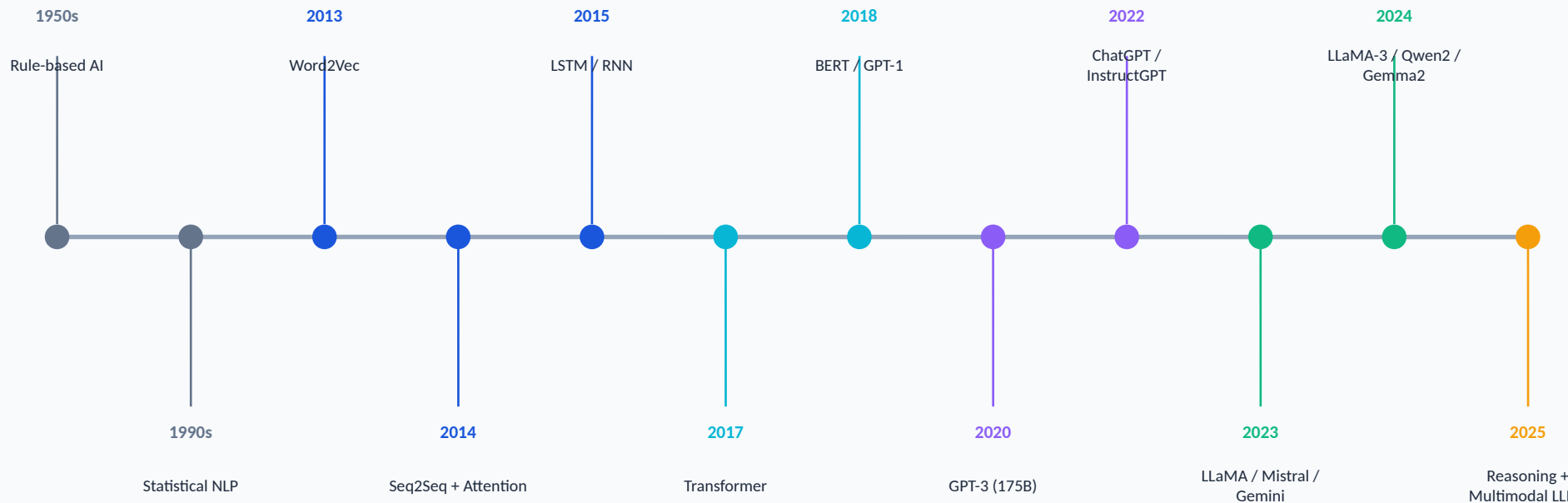
\$4B+

Invested in AI startups (2024)

100+

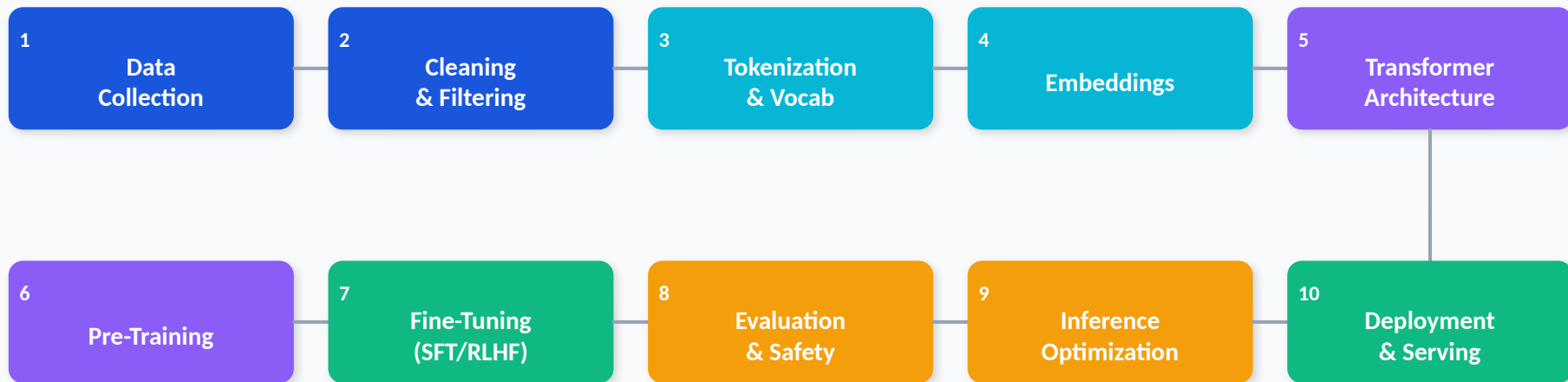
Open-source LLMs released
(2024)

History of Language Models



Key Insight: The Transformer (2017) was the watershed moment — enabling parallelization, long-range dependencies, and massive scale.

Overall Pipeline: Building an LLM from Scratch



Data Phase

Architecture Phase

Post-Training Phase

Deployment Phase

Collecting Massive Datasets

Modern LLMs train on trillions of tokens drawn from diverse, high-quality sources:

Common Crawl

Petabyte-scale web crawl data, ~3.2T tokens.
Needs heavy filtering.

Books (Books3 / Project Gutenberg)

Long-form coherent text; improves reasoning & style.

Wikipedia

Multi-language factual content; high quality, dense knowledge.

Code (GitHub, StackOverflow)

Crucial for reasoning; The Stack v2 ~900B tokens.

Research Papers (arXiv, S2ORC)

Technical depth; scientific reasoning and terminology.

Open Datasets

The Pile, FineWeb, RedPajama-v2, Dolma, SlimPajama, ROOTS.



Scale examples: LLaMA-3 ~15T tokens | Mistral Large ~unknown | Qwen2-72B ~7T tokens | FineWeb = 15T tokens from Common Crawl

Data Cleaning & Preprocessing

1 Deduplication

MinHash LSH, exact-match dedup. Removes 30-70% of web crawl data. Critical to prevent memorization.

2 Language Filtering

fastText language ID. Keep target languages; remove low-resource noise. LLaMA keeps 89% English for base.

3 Quality Filtering

Perplexity scoring, n-gram filters, heuristic rules. Remove spam, boilerplate, SEO content.

4 Safety / PII Removal

Remove phone numbers, emails, SSNs, addresses. Regex + classifier-based detection. GDPR compliance.

5 Content Filtering

Adult content, violence, hate speech classifiers. Often use models fine-tuned for safety classification.

6 Format Normalization

Strip HTML, Markdown clean-up, Unicode normalization (NFC), whitespace normalization.

Dataset Organization & Formats

Storage Formats

Format	Use Case	Library
JSONL	Instruction/chat datasets	json, datasets
Parquet	Large-scale efficient storage	pandas, pyarrow
Arrow / Feather	In-memory fast access	HuggingFace datasets
CSV / TSV	Simple tabular data	pandas, csv
Raw Text (.txt)	Pre-training corpora	streaming

Chat Template (ChatML format)

```
<|im_start|>system
You are a helpful assistant.
<|im_end|>
<|im_start|>user
What is a transformer?
<|im_end|>
<|im_start|>assistant
A Transformer is a neural network...
<|im_end|>
```

Instruction Dataset Example (Alpaca format)

```
{"instruction": "Summarize this article", "input": "Transformers were introduced in...", "output": "The Transformer architecture, introduced in 2017..."}
```

What is Tokenization?

Tokenization converts raw text into discrete units (tokens) that a model can process numerically.

Input Text:

"Transformers are amazing!"

Word-level:

"Transformers"

"are"

"amazing!"

→ Fails on unknown words (OOV problem)

BPE Sub-word:

"_Trans"

"form"

"ers"

"_are"

"_amaz"

"ing"

"!"

→ ✓ Modern standard — no OOV, efficient vocabulary

Character-level:

"T"

"r"

"a"

"n"

"s"

"..."

→ No OOV but very long sequences

Token IDs: "_Trans" → 13510 | "form" → 689 | "ers" → 388 | (Each token maps to a unique integer in the vocabulary)

Modern Tokenizers

BPE (Byte Pair Encoding)

Used by: GPT-2, GPT-3, GPT-4, LLaMA, Mistral

✓ No OOV, compact vocab, language-agnostic at byte level

✗ Greedy merging can be suboptimal

WordPiece

Used by: BERT, DistilBERT, RoBERTa

✓ Maximizes language model likelihood

✗ Requires ## prefix for continuations

SentencePiece

Used by: T5, ALBERT, Gemma, LLaMA (wraps BPE/Unigram)

✓ Language-agnostic, works without pre-tokenization

✗ Slightly more complex training

Unigram LM

Used by: XLNet, some multilingual models

✓ Probabilistic; considers multiple segmentations

✗ Slower training than BPE

TikToken (BPE variant)

Used by: GPT-3.5, GPT-4, Claude series (cl100k_base, o200k_base) ✓ Very fast Rust implementation; 100K vocab handles code well

Embeddings: Turning Tokens into Vectors

An embedding maps each token ID to a dense vector of real numbers, capturing semantic meaning.

Token Embedding

Lookup table $E \in \mathbb{R}^{(V \times d_{\text{model}})}$. Each token ID maps to a d -dimensional vector. Learned during training.

```
token_id=689 → [0.23, -0.87, 0.41, ...]
```

Positional Encoding

Adds position info since Attention has no order. Sinusoidal (original) or learned (GPT) or RoPE (LLaMA).

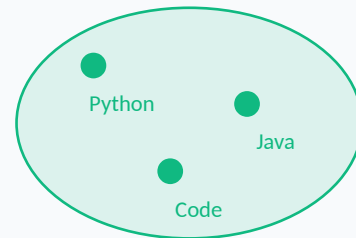
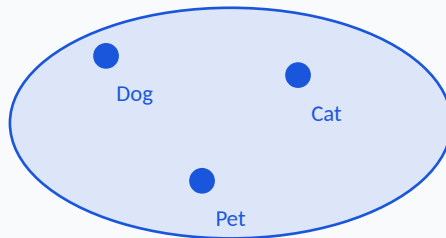
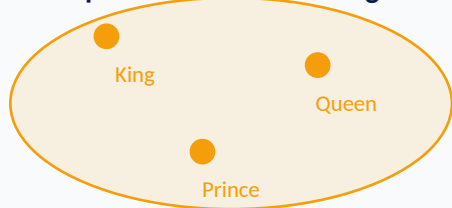
```
pos=3 → [sin(3/10000^0),  
cos(3/10000^0), ...]
```

Combined Input

$x = \text{Token_Embed}(\text{token}) + \text{Pos_Embed}(\text{position})$.
This is what enters the Transformer layers.

```
x_3 = E[token_id] + PE[3]    shape:  
(d_model,)
```

Semantic Space — Similar meanings cluster together:



Why Transformers? The Failures of RNN & LSTM

✗ RNN Problems

Sequential processing: can't parallelize over tokens

Vanishing gradients over long sequences

Forgets long-range dependencies (context > 50 tokens degrades)

$O(n)$ sequential steps = slow training on GPUs

⚠ LSTM Improvements (But Still Limited)

Gated memory cells partially solve vanishing gradients

Still sequential — no GPU parallelization

Struggles with very long documents (books, code bases)

Complex architecture, hard to scale

✓ Transformer Solutions

Fully parallel: process all tokens simultaneously

Global attention: every token attends to every other token

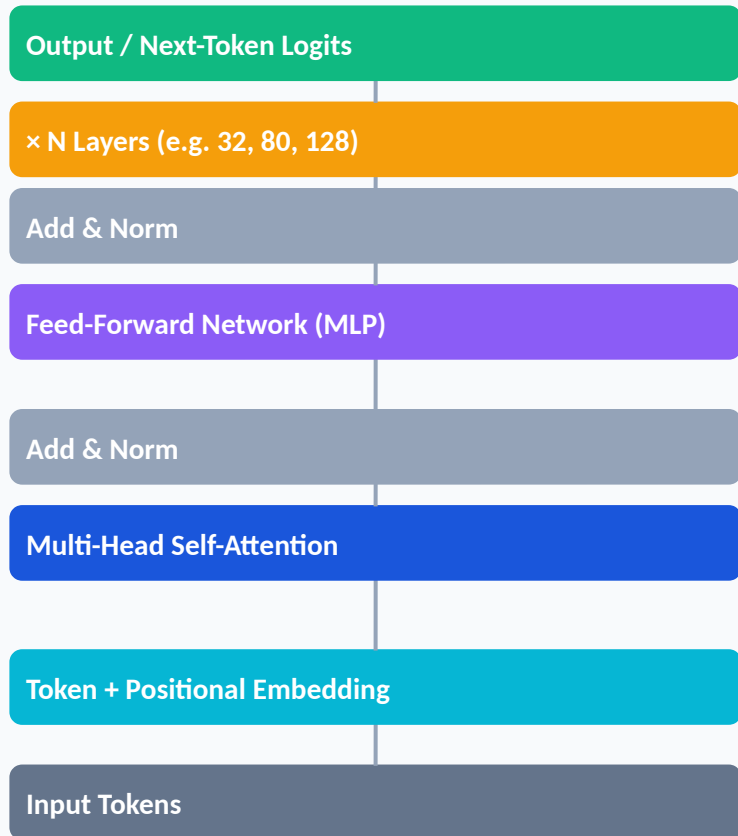
$O(1)$ path length between any two positions

Scales to billions of parameters on GPU clusters



"Attention Is All You Need" — Vaswani et al., 2017 (Google Brain). The single most cited AI paper. Replaced recurrence with self-attention entirely.

Transformer Architecture Overview



LLM Architecture Variants

Model	Params	Layers	d_model	Heads
GPT-2 Small	117M	12	768	12
LLaMA-3 8B	8B	32	4096	32
LLaMA-3 70B	70B	80	8192	64
Mistral 7B	7B	32	4096	32
Qwen2-72B	72B	80	8192	64
GPT-3 (public)	175B	96	12288	96

Key Hyperparameters:

d_model = embedding dim | n_heads = attention heads | n_layers = transformer depth | d_ff = FFN hidden dim ($\approx 4 \times d_{\text{model}}$)

The Attention Mechanism

Attention lets every token ask: 'Which other tokens are most relevant to understanding me?'



Real-World Analogy

Sentence: "The animal didn't cross the street because it was too tired."

Question: What does "it" refer to?

Attention: "it" attends strongly to "animal" (0.8) and weakly to "street" (0.1).

The model learns these weights from data.

Attention Weight Heatmap (simplified)

"it" →	0.1	0.8	0	0	0.1
	The	animal	didn't	it	tired

Why Attention Works:

- Captures long-range dependencies (token 1 can directly attend to token 5000)
- Learns context-dependent representations (same word, different meaning → different attention)
- Parallelizable: all attention computations happen simultaneously on GPU
- Interpretable: attention weights reveal what the model 'focuses on'

Query, Key, and Value

Attention uses three learned linear projections of the input embeddings: Q, K, and V.

Library / Search Engine Analogy

Query (Q): What you're searching for. 'Find me information about machine learning.'

Key (K): Index/title of each document. 'Chapter 3: Neural Networks'.

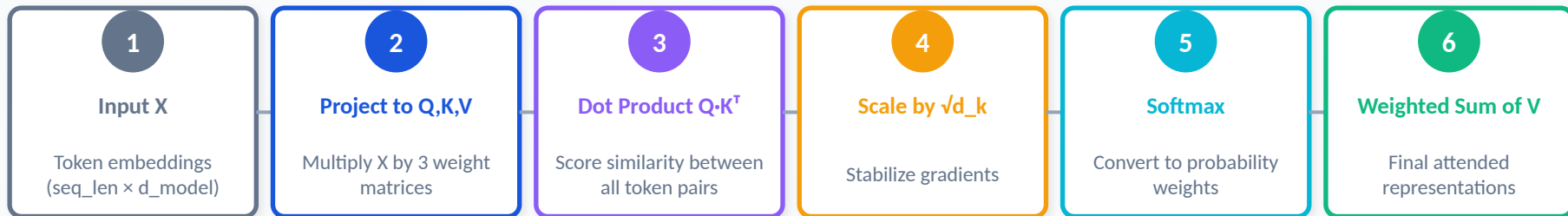
Value (V): Actual content of each document. The knowledge to retrieve.

Linear Projections

$$\begin{aligned} Q &= X \cdot W_Q & (d_{\text{model}} \times d_k) \\ K &= X \cdot W_K & (d_{\text{model}} \times d_k) \\ V &= X \cdot W_V & (d_{\text{model}} \times d_v) \end{aligned}$$

Where X is input, W are learned weights

Step-by-step QKV Calculation:



Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}} \right) \cdot V$$

Numerical Walkthrough (2 tokens, $d_k = 2$):

Step 1 — Q and K matrices:

$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

Step 2 — Dot product $Q \cdot K^T$:

$\begin{bmatrix} 1 \cdot 1 + 0 \cdot 0 & 1 \cdot 0 + 0 \cdot 1 \\ 0 \cdot 1 + 1 \cdot 0 & 0 \cdot 0 + 1 \cdot 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

Step 3 — Scale by $\sqrt{d_k} = \sqrt{2} \approx 1.41$:

$\begin{bmatrix} 0.71 & 0.0 \\ 0.0 & 0.71 \end{bmatrix}$

Step 4 — Softmax per row:

$\begin{bmatrix} 0.67 & 0.33 \\ 0.33 & 0.67 \end{bmatrix}$

Step 5 — Multiply by V : weighted sum output

What Each Component Does:

$Q \cdot K^T$

Computes raw similarity score between every pair of tokens. Higher dot product = more similar = more attention.

$\frac{\cdot}{\sqrt{d_k}}$

Scaling prevents dot products from growing too large (which pushes softmax into near-zero gradient regions).

$\text{softmax}(\cdot)$

Converts raw scores to positive probabilities summing to 1. Creates attention distribution.

$\cdot V$

Weighted sum: retrieve information from Value vectors, weighted by how much attention to pay.

⚡ Complexity: $O(n^2 \cdot d)$ in time and $O(n^2)$ in memory — the quadratic bottleneck. Flash Attention (Dao, 2022) resolves this with block-wise IO-aware computation.

The Softmax Function

Softmax converts raw scores (logits) into a probability distribution over all tokens.

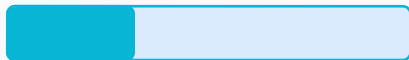
$$\text{softmax}(\mathbf{x}_i) = \exp(\mathbf{x}_i) / \sum_j \exp(\mathbf{x}_j) \quad \rightarrow \quad \text{All outputs} \geq 0$$

and $\Sigma = 1.0$

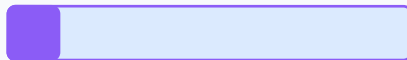
Example:



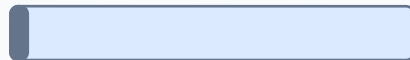
cat: logit=2 $\rightarrow$ softmax=0.638



dog: logit=1 $\rightarrow$ softmax=0.235



fish: logit=0.1 $\rightarrow$ softmax=0.095



bird: logit=-1 $\rightarrow$ softmax=0.032

Sum of all softmax values = 1.000 ✓

Key Properties of Softmax:

- Exponential: amplifies differences — the highest logit dominates the distribution
- Differentiable: gradients flow through softmax for backpropagation
- Temperature scaling: $\text{softmax}(\mathbf{x}/T)$ with $T > 1$ makes distribution flatter (diverse outputs), $T < 1$ makes it sharper (greedy)
- Used in two places: (1) attention weights inside transformer (2) final output token probability distribution

Multi-Head Attention

Instead of one attention function, run H parallel attention heads, each learning different relationships.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W_O \quad \text{where } \text{head}_i = \text{Attention}(Q \cdot W_{i_Q}, K \cdot W_{i_K}, V \cdot W_{i_V})$$

Head 1

Syntactic
relations

Head 2

Coreference
resolution

Head 3

Semantic
similarity

Head 4

Local
context

Head 5

Long-range
dep.

Head 6

Head 6
(other)

↓ Concatenate all head outputs → W_O linear projection → Final MHA output

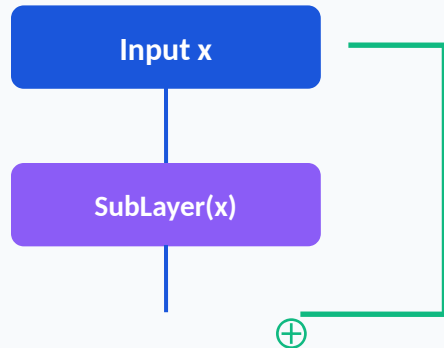
💡 GPT-3 uses 96 heads, $d_{\text{model}}=12288$, $d_k = 128$ per head. Each head specializes in different linguistic/structural patterns — this is empirically verified by Attention Head Analysis research (Clark et al., 2019).

Residual Connections & Layer Normalization

The Problem: Vanishing Gradients

- In deep networks (32-128 layers), gradients shrink exponentially as they flow backward.
- After many multiplications < 1 , gradient $\approx 0 \rightarrow$ weights stop updating.
- Result: lower layers learn nothing — training fails.

Residual Connection (Skip Connection)



Layer Normalization (LayerNorm)

- Normalizes activations across the feature dimension (not batch). Stabilizes training with large learning rates.
- Pre-LN (LLaMA style): LayerNorm before each sub-layer — more stable, current standard.
- Post-LN (original paper): LayerNorm after residual — less stable for deep models.
- RMSNorm (LLaMA, Mistral): Simplified version — normalizes by RMS, no mean subtraction. ~10% faster

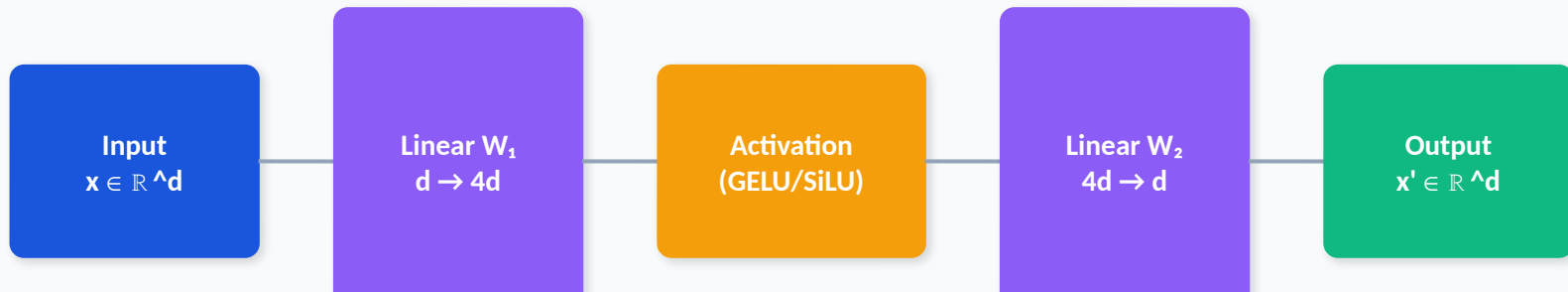
$$\text{LayerNorm}(x) = \gamma \cdot (x - \mu) / \sqrt{\sigma^2 + \epsilon} + \beta$$
$$\text{RMSNorm}(x) = x / \text{RMS}(x) \cdot \gamma \quad \text{where } \text{RMS} = \sqrt{(\sum x_i^2 / d)}$$



Residual connections were key to training ResNet (2015, He et al.). GPT uses both residual connections AND layer norm in every block — they work synergistically.

Feed-Forward Network (FFN / MLP)

After attention, each token independently passes through a 2-layer MLP — the 'thinking' layer.



$$\text{FFN}(x) = W_2 \cdot \text{GELU}(W_1 x + b_1) + b_2 \quad d_{\text{ff}} \approx 4 \times d_{\text{model}} \quad (\text{e.g., } d_{\text{model}}=4096 \rightarrow d_{\text{ff}}=16384 \text{ for 7B models})$$

Activation Functions:

ReLU — $\max(0, x)$

Used: Original Transformer, GPT-2

GELU — $x \cdot \Phi(x)$

Used: BERT, GPT-3, most modern LLMs

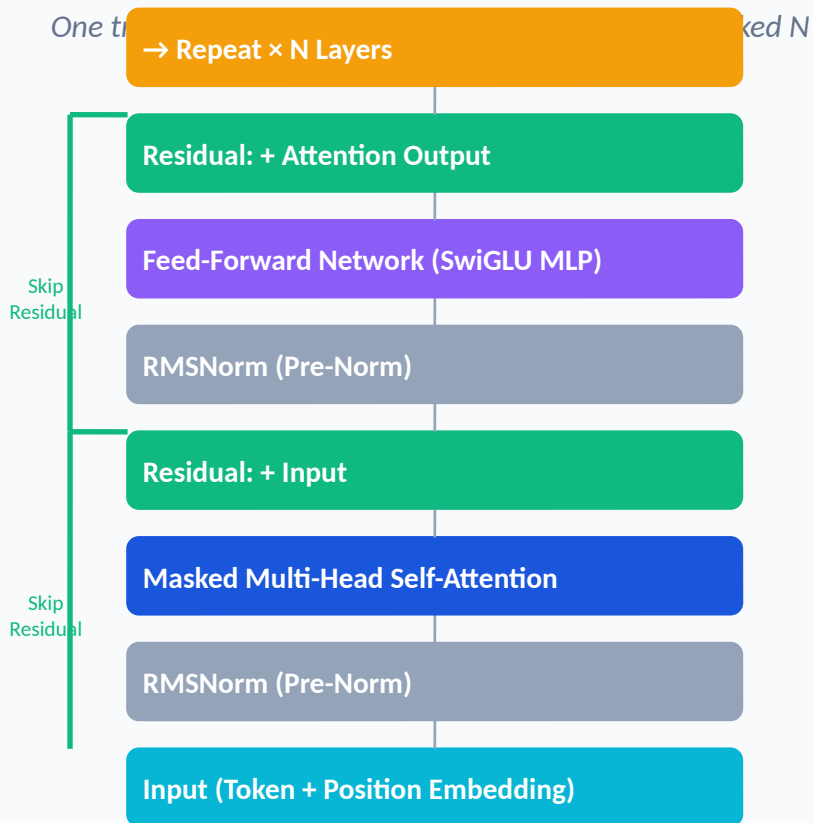
SiLU / Swish — $x \cdot \sigma(x)$

Used: LLaMA, Mistral, Qwen2, Gemma

GeGLU / SwiGLU — $(W_1 x) \odot \text{SiLU}(W_2 x)$

Used: PaLM, LLaMA-3, GPT-4 (likely)

The Complete Transformer Block



Pre-Norm (LLaMA)

Apply RMSNorm to input BEFORE the sub-layer. More training stable.

Causal Mask

In training: future tokens are masked. Enables autoregressive learning.

Two Residuals

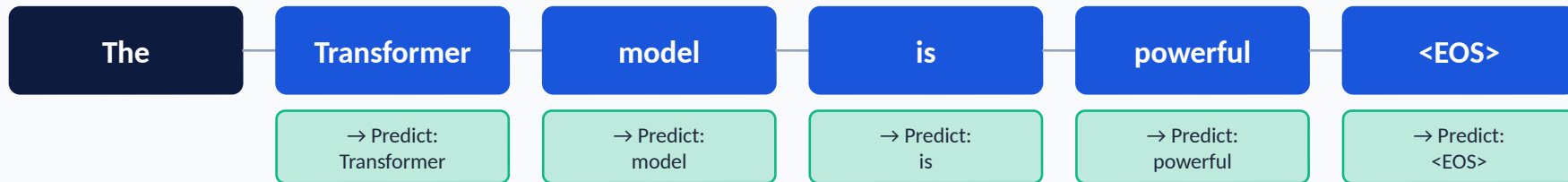
One wraps MHA, one wraps FFN. Both critical for gradient flow.

Layer Count

7B → 32 layers. 70B → 80 layers. Each layer adds ~6M-150M parameters.

Training Objective: Next Token Prediction

The fundamental task: given all previous tokens, predict the next one. Repeated across trillions of tokens.



Teacher Forcing: during training, always feed the GROUND TRUTH token (not the model's prediction) as the next input.

Training Loop (Pseudo-code):

```
for batch in dataloader:
    tokens = batch["input_ids"]      # (B, T)
    inputs  = tokens[:, :-1]         # all but last
    targets = tokens[:, 1:]          # all but first

    logits = model(inputs)           # (B, T, vocab_size)
    loss = cross_entropy(logits, targets)

    optimizer.zero_grad()
    loss.backward()
```



A 7B LLM trained on 2T tokens processes and learns from ~2 trillion next-token prediction examples. Training loss is averaged Cross Entropy over all positions in all batches.

Why Next Token Prediction Works:

- Self-supervised: no human labels needed — the text itself provides infinite training signal.
- Emergent understanding: to predict well, the model must understand grammar, facts, logic, and context.

Loss Function: Cross-Entropy

Cross-Entropy Loss measures how different the model's predicted probability distribution is from the true next token.

$$L = - (1/N) \cdot \sum_i \log(P_{\text{model}}(t_i \mid t_1 \dots t_{i-1}))$$

Intuition — 3 Cases:

Confident & Correct

Model predicted: P=0.95 for 'cat'

$$\text{loss} = -\log(0.95) = 0.05 \quad \checkmark \text{ LOW}$$

Confident & Wrong

Model predicted: P=0.05 for 'cat'

$$\text{loss} = -\log(0.05) = 3.0 \quad \times \text{ HIGH}$$

Uncertain (random)

Model predicted: P=0.5 for 'cat'

$$\text{loss} = -\log(0.5) = 0.69 \quad \sim \text{MID}$$

Perplexity (PPL) = $\exp(\text{CrossEntropyLoss})$

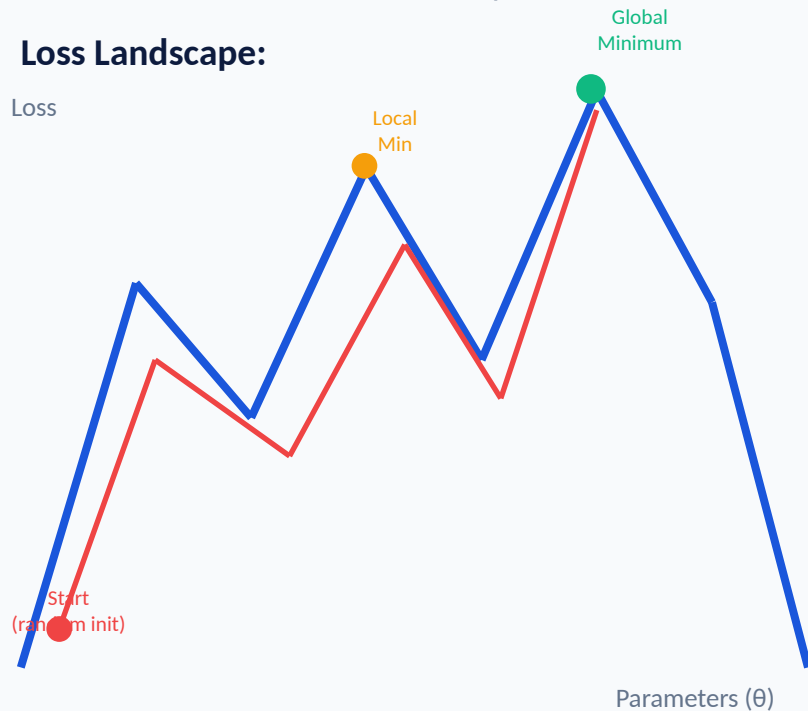
- PPL = 1.0: perfect predictions. PPL = V (vocab size ≈ 32000): random guessing.
- GPT-2 (1.5B): PPL ≈ 18 on WikiText. LLaMA-3 8B: PPL ≈ 6 on standard benchmarks.

HERE AND NOW AI | Author's Experience | How LLMs Are Built | Every doubling of training compute typically lowers PPL by ~ 1 -2 points.

Gradient Descent & Optimization

Gradient Descent finds the model parameters that minimize the loss function — the 'downhill walk'.

Loss Landscape:



Gradient:

$\partial L / \partial \theta$ — direction of steepest INCREASE in loss. We move OPPOSITE direction.

Learning Rate (η):

Step size. Too large → overshoot. Too small → slow convergence. Typical: $1e-4$ to $3e-4$.

Mini-Batch SGD:

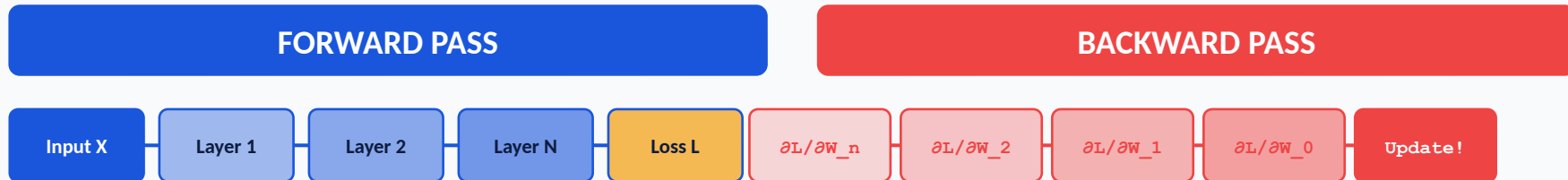
Compute gradient over small batches (e.g., 2M tokens). Noise helps escape local minima.

Weight Update:

$$\theta \leftarrow \theta - \eta \cdot \partial L / \partial \theta$$

Backpropagation

Backpropagation computes gradients for every weight in the network using the chain rule of calculus.



Chain Rule:

$\partial L / \partial W_1 = \partial L / \partial a_2 \cdot \partial a_2 / \partial a_1 \cdot \partial a_1 / \partial W_1$ (chain rule — multiply local gradients)

PyTorch Autograd

```
loss = model(inputs) vs targets
loss.backward() # compute all gradients
optimizer.step() # update all weights
```

Key Backpropagation Facts:

- Gradient memory: backward pass requires storing all intermediate activations from forward pass. A 70B model backward pass uses $\sim 2\times$ more GPU memory than forward.
- Gradient checkpointing: recomputes activations during backward instead of storing — trades compute for memory (used when training large models).
- Gradient clipping: clip gradient norm to max 1.0 to prevent explosive gradient updates early in training.

Optimizers: From SGD to AdamW

SGD + Momentum

```

$$\begin{aligned}v &\leftarrow \beta \cdot v + \eta \cdot g \\ \theta &\leftarrow \theta + v\end{aligned}$$

```

Adds velocity. Faster convergence than plain SGD. Poor for sparse gradients. Rarely used for LLMs.

Used by: Classic CV models

RMSProp

```

$$\begin{aligned}E[g^2] &\leftarrow \beta \cdot E[g^2] + (1-\beta) \cdot g^2 \\ \theta &\leftarrow \theta - \eta \cdot g / \sqrt{E[g^2] + \epsilon}\end{aligned}$$

```

Adaptive learning rate per parameter. Divides by RMS of recent gradients. Unstable for LLM scale.

Used by: RNNs, early models

Adam

```

$$\begin{aligned}m &\leftarrow \beta_1 m + (1-\beta_1) g \\ v &\leftarrow \beta_2 v + (1-\beta_2) g^2 \\ \theta &\leftarrow \theta - \eta \cdot m / \sqrt{v} + \epsilon\end{aligned}$$

```

Adaptive learning rates + momentum. Combines RMSProp and Momentum. The standard for LLMs.

Used by: BERT, most LLMs

AdamW ✓ (Current Standard)

```

$$\begin{aligned}\theta &\leftarrow \theta(1 - \lambda) - \eta \cdot m / (\sqrt{v} + \epsilon) \\ &\text{(L2 weight decay applied correctly)}\end{aligned}$$

```

Decouples weight decay from gradient update. Fixes Adam's L2 regularization bug. Used by LLaMA, Mistral, Qwen, GPT-4 (likely).

Used by: GPT-3, LLaMA, all modern LLMs

Learning Rate Schedule: Warmup (linear, ~2K steps) → Peak lr (e.g., 3e-4) → Cosine decay to 1e-5 over training. Note: Exact optimizer details for GPT-4, Claude, and Gemini are proprietary — AdamW is the known industry standard.

Training Infrastructure: GPU Clusters & Distributed Training

Hardware

NVIDIA H100 80GB

~3,958 TFLOPS BF16

NVIDIA A100 80GB

~624 TFLOPS BF16

Google TPU v4 Pod

1.1 EFLOPS (1152 chips)

AMD MI300X 192GB

~1,307 TFLOPS FP16

Parallelism Strategies

Data Parallelism

Each GPU processes different mini-batches. Gradients averaged across GPUs. Simple. (DDP)

Tensor Parallelism

Split weight matrices across GPUs (Megatron-LM). Each GPU holds a shard of each layer.

Pipeline Parallelism

Different transformer layers on different GPUs. Micro-batches fill the pipeline to reduce idle time.

FSDP / ZeRO

Shard optimizer states, gradients, and parameters across GPUs. ZeRO-3 reduces memory by #GPUs $\times$.

Mixed Precision

BF16

Brain Float 16. Same exponent range as FP32. Standard for LLM training. No overflow risk.

FP16

Half precision. Faster but narrower exponent range $\rightarrow$ overflow risk. Requires loss scaling.

FP8

New: H100 supports FP8 (E4M3/E5M2). $\sim 2\times$ speedup over BF16. Used in some training (Llama-3).

FP32

Master weights and optimizer states kept in FP32 for numerical stability.

⚡ Scale: LLaMA-3 70B trained on 15T tokens using ~16,000 H100 GPUs for ~6 months = 6.4M GPU-hours $\approx$ \$30M+ compute cost. 3D parallelism (data + tensor + pipeline) is required at this scale.

Post-Training: Fine-Tuning for Instructions & Alignment

A pre-trained model predicts next tokens well but doesn't follow instructions. Post-training aligns it with human intent.

1 Supervised Fine-Tuning (SFT)

Train on (instruction, response) pairs. Dataset: ~10K-500K examples. Models: LLaMA, Alpaca, Vicuna.
Method: same cross-entropy loss but only on the RESPONSE tokens.

2 RLHF — Reward Model

Human annotators rank multiple model outputs. Train a Reward Model (RM) to predict human preference scores. Used in InstructGPT, ChatGPT, Claude.

3 RLHF — PPO Fine-Tuning

Use PPO (Proximal Policy Optimization) RL algorithm to optimize model outputs toward the reward model. Complex and expensive.

4 DPO (Direct Preference Optimization)

Bypass reward model. Directly train on (preferred, rejected) response pairs. Simpler, more stable than PPO. Now dominant (LLaMA-3, Mistral, Qwen2).

5 Constitutional AI (Anthropic)

Claude's alignment approach. Uses AI feedback (RLAIF) instead of human feedback. Model critiques & revises its own outputs using a set of principles (constitution).

6 Parameter-Efficient Fine-Tuning (PEFT)

LoRA: add small low-rank matrices (rank 4-64) beside frozen weights. Only 0.1-1% of parameters. QLoRA: LoRA on 4-bit quantized model. Runs on a single GPU.

Inference Optimization

Serving LLMs efficiently is as challenging as training. These techniques make production inference feasible.

KV Cache

Cache Key/Value matrices from previous tokens. Avoids recomputation. $\sim 10\times$ decode speedup. Memory cost: $O(n_{\text{layers}} \times \text{seq_len} \times 2 \times d_{\text{head}} \times n_{\text{heads}})$.

Flash Attention

Tri Dao, 2022/2023. Block-wise, IO-aware attention. $O(n)$ memory, 2-4 $\times$ faster. Standard in all modern inference engines.

Quantization (GPTQ / AWQ)

Reduce model weights to INT8, INT4, or INT3. 4-bit: $\sim 4\times$ memory reduction with $< 1\%$ accuracy loss. Enables 70B models on $2\times$ 48GB GPUs.

Speculative Decoding

Use a small 'draft' model to generate token candidates; verify with large model in parallel. 2-3 $\times$ decode speedup. Used in production at Google, Anthropic.

Continuous Batching (vLLM)

PagedAttention (Kwon et al., 2023). Dynamic batching of requests. 23 $\times$ higher throughput than HuggingFace Transformers naive serving.

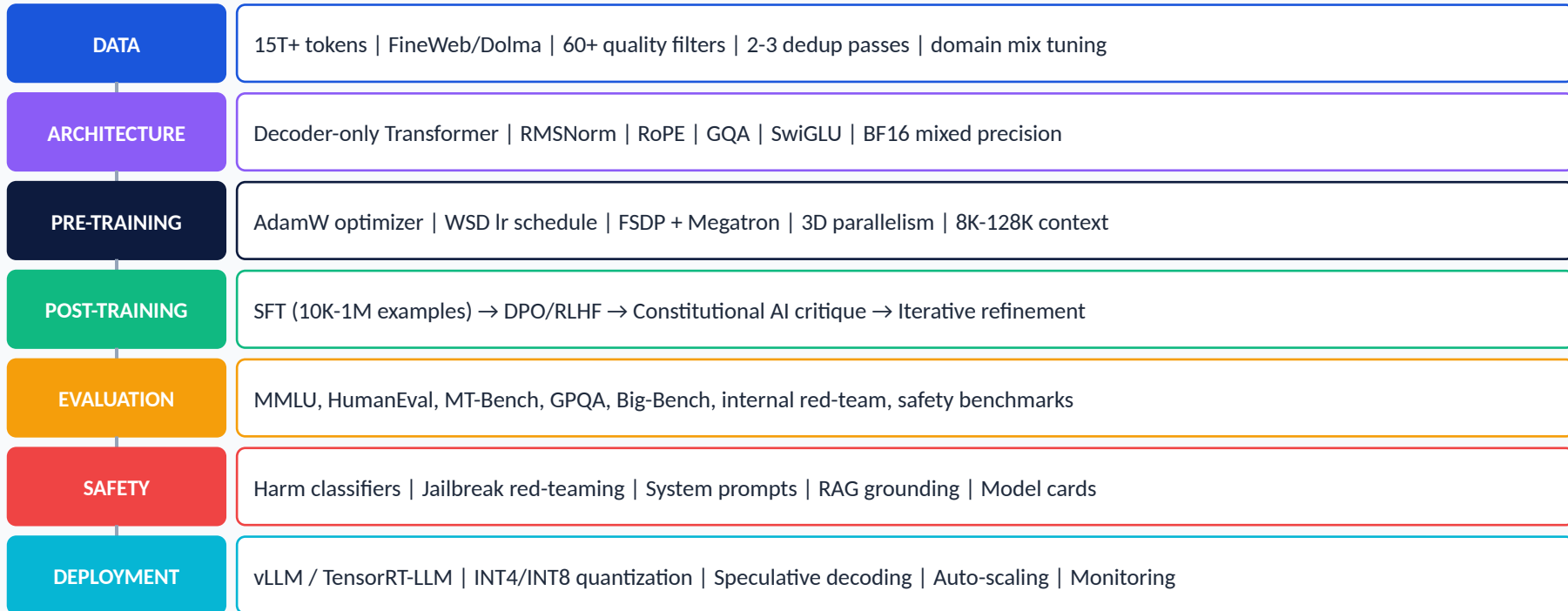
Pruning & Distillation

Pruning: remove low-importance weights. Distillation: train small model to mimic large (DistilBERT 40% smaller, 60% faster). MiniCPM, Phi are distilled models.

Key Frameworks: vLLM (OSS, production-grade) · TensorRT-LLM (NVIDIA, max GPU throughput) · llama.cpp (CPU/mobile) · Ollama (local, consumer HW) · SGLang (structured gen)

How Modern LLMs Are Built Today: Full Production Pipeline

The end-to-end pipeline followed by leading AI organizations (based on publicly available information from Meta, Mistral AI, Qwen team, Google DeepMind, 2024-2025).



⚠️ Proprietary: Exact training data mixture, optimizer details, evaluation thresholds, and safety systems for GPT-4, Claude 3/4, and Gemini Ultra are not publicly disclosed. This slide reflects open-source equivalents.

Summary & The Future of LLMs

① Data

Trillion tokens, filtered & deduplicated

② Tokenize

BPE / SentencePiece → integer sequences

③ Embed

Token + RoPE positional embeddings

④ Transform

$N \times [\text{Attention} + \text{FFN} + \text{Residuals}]$

⑤ Train

NTP loss, AdamW, backprop, 10K+ GPUs

⑥ Align

SFT → DPO/RLHF → Safety filters

Future Trends (2025–2027)

Mixture of Experts (MoE)

Sparse models: only 2-8 experts (sub-FFNs) activate per token. $8\times$ more parameters with same compute. GPT-4, Mixtral, Qwen-MoE use MoE.

Reasoning Models

Chain-of-thought at scale: o1/o3 (OpenAI), DeepSeek-R1. Think longer before answering. Test-time compute scaling.

Long-Context Models

128K-1M token contexts. Gemini 1.5 (1M), Claude 3.7 (200K). Sparse attention & chunk-wise processing.

Multimodal AI

Vision + Audio + Video + Text in one model. GPT-4o, Gemini Ultra, LLaMA-3.2 Vision. End-to-end native multimodal.

Agentic AI

LLMs that use tools, browse the web, write & run code, plan multi-step tasks. Anthropic Computer Use, Claude Code, OpenAI Operator.

Efficient Models

Sub-1B to 7B models matching 70B capability. Knowledge distillation, architectural innovations. Phi-4, MiniCPM, Gemma 2 2B.



Key Takeaway: LLMs are not magic — they are engineering. $\text{Data} \times \text{Scale} \times \text{Architecture} \times \text{Alignment} = \text{Intelligence}$. |

www.ruthranraghavan.com